

AI Infra · 算子与系统

从大规模 hang 到通信 Kernel：最小复现、开关二分与回归闭环

CUDA Graph

Collective

最小复现

性能回归

岗位能力画像

岗位画像：会把稳定性故障缩小到可控范围，也能解释通信、图捕获与算子融合的相互约束。

高频深挖：题目背后的判断

为什么 Graph on 才卡住？

先固定 graph on/off，再缩并行度、fused 路径和 fallback，而不是先猜代码行。

开关矩阵把大故障变成一个可证伪的假设。

通信 kernel 为什么会忙等？

区分 payload 与 control state；控制字若用近似比较或截断编码，合法值可能被误判。

状态机要有类型、可见性和退出条件。

通信算法能否背结论？

Ring/Tree 的选择必须回到消息、拓扑、延迟、带宽和 overlap。

先写工作负载，再谈通信原语。

修复后怎样证明不是偶然？

最小复现、负向控制、不同并行度、重复 replay 与端到端回归共同成立。

修复 = 触发条件被覆盖，而非一次不复现。

项目叙事的通用深挖链

问题与约束

证据与机制

方案与取舍

验证与边界

答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实

现象

固定 workload / 阶段 / 指标，明确‘慢’或‘错’在哪里。

证据

用 trace、counter、对照或最小复现排除相邻假设。

机制

说明资源、数据流或依赖如何导致这个现象。

验收

数值、边界、重复性和 E2E 共同成立后再下结论。

一题答透：从 hang 到根因：控制变量不是装饰

问题 “一个多卡图执行偶发卡住，如何把它缩到通信相关的 kernel 级问题？”

1. 缩问题

缩请求、缩 batch、缩并行度；保留触发现象，删除与现象无关的模型复杂度。

2. 做矩阵

Graph on/off、并行度、fused collective、fallback 四类开关逐次二分，不混改。

3. 读状态机

确认 producer/consumer、flag 可见性、等待条件和退出条件；区分控制状态与正常数据。

4. 建回归

把最小复现变成持续回归，覆盖重复 replay、不同 shape 和失败路径。

把推理链画出来



常见误区

× 直接从整模型日志猜根因，无法建立因果。

× 一次不 hang 就宣布修复。

× 只优化通信或只优化计算，却没有测真正的 stream overlap。

术语速懂 | 面试怎么答

术语速懂

CUDA Graph

记录稳定 GPU 工作并 replay，主要减少 host 提交开销；拓扑、shape、地址和生命周期需稳定。

控制状态

通信 payload 与完成 flag 不同；flag 需精确定义类型、可见性、等待与退出条件。

面试中怎么答

- ① 先用时间线确认是 launch gap / host sync，而不是长 kernel。
- ② 再用最小复现与开关矩阵把变量缩到一条通信路径。
- ③ 用 graph on/off、重复 replay 和失败路径验证修复可复现。

白板 / 手撕：白板题：定时器 / 最小堆与复杂度

题面 大量超时、重试和延迟任务怎样调度？请说明数据结构、复杂度和取消语义。

C++17 · min-heap with lazy cancellation

核心实现

```
01 struct Task { long long due; int id; uint64_t gen; };
02 struct Later { bool operator()(const Task& a,
03     const Task& b) const { return a.due > b.due; } };
04 priority_queue<Task, vector<Task>, Later> q;
05 vector<uint64_t> version(n, 0);
06 void schedule(int id, long long due) {
07     q.push({due, id, ++version[id]});
08 }
09 void cancel(int id) { ++version[id]; }
10 void run_due(long long now) {
11     while (!q.empty() && q.top().due <= now) {
12         Task t = q.top(); q.pop();
13         if (t.gen == version[t.id]) handle(t.id);
14     }
15 }
```

写代码时怎么讲

1. min-heap：peek $O(1)$ ，push/pop $O(\log n)$ ，适合稀疏 deadline。
2. 每个任务带 generation / token，取消时惰性失效，避免线性删除。
3. 短周期且密集的任务可讨论 timing wheel。
4. 追问：重试如何防重复、怎样记录尾延迟和队列积压？

实现重点

- 复杂度：peek $O(1)$ ，push/pop $O(\log n)$ 。
- generation 使旧重试/超时任务惰性失效。
- 堆中过期节点过多时需要重建或压缩。

边界、测试与追问

最少要说明的验证

- 复杂度：peek $O(1)$ ，push/pop $O(\log n)$ 。
- generation 使旧重试/超时任务惰性失效。
- 堆中过期节点过多时需要重建或压缩。

面试继续追问

- TCP/UDP 的可靠性边界是什么？
- 区间合并如何处理相接区间？
- 什么时候通信与计算 overlap 反而变慢？